

s3.04 — Strongly consistent metadata / KV store for control-plane resources

Index

- Interview framing
- Requirements — IC3 baseline
- Capacity and consequence — IC3 baseline
- Architecture and flow — IC3 baseline
- API and data model — IC3 baseline
- Quorum commit and linearizable reads — IC3 baseline
- Snapshots, watches, and rebalancing — IC3 baseline
- Failure, operations, and security
- Alternatives and IC4 stretch
- Level signal — IC3 baseline vs IC4 stretch
- Interview pushback and validation
- Recall card
- References

Interview framing

- Target: OCI IC3 ownership of a bounded platform component.
 - IC3 baseline is what must land in every phase.
 - IC4 stretch changes the architecture only when a stronger requirement earns the cost.
 - These are preparation labels, not Oracle's official rubric.
- Evidence: ranked bank.
 - Prior supplemental bank, adapted to control-plane metadata.
 - This is a proposed design, not a claim about OCI internals or a reported accepted answer.
- Why this is hard
 - Naive: replicate writes to several independent nodes and choose the latest timestamp on read.
 - First break: two clients both succeed at a conditional ownership update during a partition.
 - Second break: a partitioned old leader keeps answering reads from local state, so a client sees a value that a completed write already replaced.
 - Minimum primitives
 - A replicated state machine per shard.
 - Linearizable reads and compare-and-set.
 - Durable log and snapshots.
 - Authoritative shard ownership and safe membership change.
- 🗣️ "Strong metadata needs one agreed order of operations, not just overlapping read and write replicas."
- Opening script
 - "I will build a regional metadata store with linearizable reads and conditional writes."
 - "I will confirm whether transactions need to span shards before choosing the partition key."
 - "Each shard will use a three-replica Raft group and a deterministic state machine."
 - "The deep dive is why a newly elected leader can safely answer a read and how a timed-out write is retried."
 - "I will close with quorum loss, snapshots, and the cost of cross-region consistency."

Phase	Clock	Deliverable
Intro	00:00–05:00	Establish role context and confirm the prompt

Phase	Clock	Deliverable
Requirements	05:00–11:00	Scope and one correctness invariant
Capacity and API	11:00–17:00	A number that forces a choice; retry contract
Architecture	17:00–30:00	Draw ownership and trace one request
Deep dive	30:00–45:00	Concurrency proof and a crash boundary
Operations and alternatives	45:00–50:00	Degradation, recovery, and flip condition
Wrap-up	50:00–54:00	Four-sentence recap and unresolved assumption
Interviewer questions	54:00–60:00	Team ownership and production priorities

- Clock note
 - The design slice is 45 minutes within the 60-minute round.
 - If the interviewer gives 45 minutes total, use a 2-minute intro and compress the remaining phases.
 - The interviewer's clock wins.

Requirements — IC3 baseline

- Numerical targets are interview assumptions unless an adjacent primary source states otherwise.

Question	Assumed answer	Why it changes the design
What does strongly consistent mean?	Linearizable Get, Put, Delete, and CAS	A successful completed write must precede a later read
Transaction scope?	Keys within one shard	Avoid hiding a distributed transaction protocol
Availability during partition?	Only the quorum side serves authoritative operations	Safety takes priority over minority availability
Data size?	Small metadata values, maximum 64 KB	Large blobs belong in object storage
Watch semantics?	Ordered revision stream with resnapshot after compaction	A watch is not a linearizable read

- Functional scope
 - Get and put bounded metadata values.
 - Compare revisions and atomically mutate keys within one shard.
 - List a range at a fixed revision.
 - Watch changes from a revision.
 - Restore snapshots and rebalance replica membership.
- Targets below are interview assumptions, not product SLAs.

Requirement	Target	Enforced by
Read latency	p99 < 10 ms regional assumption	Leader authority check and local applied state
Write latency	p99 < 20 ms regional assumption	Durable quorum replication
Availability	99.9% target with any 1 replica unavailable	3 replicas across independent fault domains
Acknowledged write loss	0 under one-replica loss	Durable majority log before acknowledgment
Value size	Maximum 64 KB	Request validation and transaction byte limits
Watch history	1 h retained baseline	Revision log with explicit compaction response

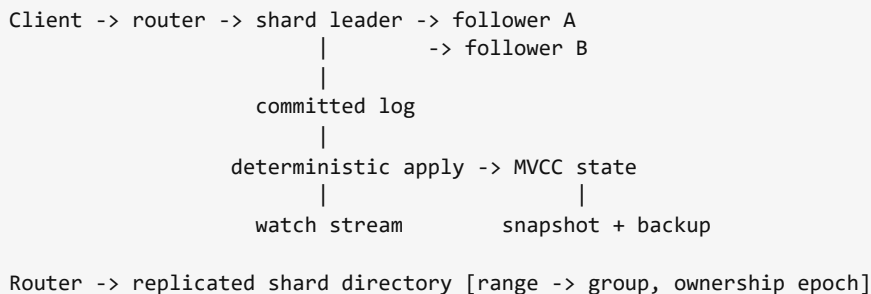
- Invariants
 - A completed successful mutation has one position in the committed shard log.
 - A linearizable read observes at least all writes completed before it began.
 - At most one CAS comparing a given expected revision can win; every later CAS observes the changed revision.
 - Only committed entries mutate externally visible state.

- A shard ownership change cannot create two independently writable copies.
- Non-goals
 - Availability on the minority side of a partition.
 - Global transactions across arbitrary shards.
 - Leaderless last-write-wins as a substitute for linearizability.

Capacity and consequence — IC3 baseline

- All workload values below are sizing assumptions.
- Workload
 - Assume 100,000 reads/s, 10,000 writes/s, and 1 KB average values.
 - Logical write bytes are 10 MB/s before log framing.
 - Decision: measure one group's quorum/fsync limit before deciding how many shards are required.
- Stored state
 - 100M keys × 1 KB = 100 GB of values before keys and indexes.
 - Three replicas require at least 300 GB before versions and log retention.
 - Decision: shard by tenant/key range and bound historical MVCC storage.
- Watch history
 - 10,000 writes/s × 1 KB × 3,600 s = 36 GB of one-hour change payloads.
 - Decision: compact explicitly and make clients support snapshot recovery.
- Hot key
 - One key still belongs to one ordered state machine even with 100 shards.
 - Decision: remove unnecessary global counters or batch semantically compatible updates.

Architecture and flow — IC3 baseline



- Router and directory
 - Buy stable shard ownership and request routing.
 - Pay metadata caching and retry on stale routing epochs.
 - Raft group
 - Buys one agreed operation order under crash faults.
 - Pays quorum latency and minority unavailability.
 - MVCC state machine
 - Buys revision comparisons and consistent range snapshots.
 - Pays version retention and compaction.
 - Snapshot subsystem
 - Buys bounded recovery and log truncation.
 - Pays snapshot IO and consistency checks.
- Authenticate tenant and resolve the authoritative shard ownership epoch.
- Route a mutation to the current group leader.
- Append the command and replicate durably to a majority.
- Commit only under the Raft commitment rule and apply in log order.

- Return the applied revision and persisted operation result.
- For a later linearizable read, establish leader authority and wait for the required applied index.

API and data model — IC3 baseline

- Proposed API contracts.
 - Authenticate the tenant from credentials before looking up any resource.
 - Retain idempotency records for the documented retry horizon.

Surface	Contract	Idempotency and conflicts
PUT /kv/{key}	Value plus optional expectedRevision	Stable requestId deduplicates ambiguous retries; failed comparison is 409
GET /kv/{key}	Value and revision; default linearizable	Explicit stale reads require an opt-in consistency mode
POST /transactions	Compare and mutate keys within one shard	Atomic committed command; cross-shard request rejected
GET /kv?prefix=&cursor=	Cursor binds range and snapshot revision	Compacted snapshot returns a restart-required error
GET /watch?prefix=&fromRevision=	Ordered changes and revision cursor	Compacted cursor requires snapshot plus resume
DELETE /kv/{key}	Optional expectedRevision	Tombstone at a committed revision; stale comparison is 409

State	Key	Fields and invariant
KV version	(tenant, key, revision)	Value or tombstone; visible only after commit
Raft log	(groupId, logIndex)	Term and deterministic command; durable before vote/ack obligations
Request result	(tenant, clientId, requestId)	Command hash and committed result retained for retry horizon
Snapshot	(groupId, lastIncludedIndex, term)	State, dedup results, and membership metadata
Directory	(rangeStart, ownershipEpoch)	Authoritative group and transfer state

- Reference DDL
 - Keys and uniqueness match the state table above and the [companion schema](#).
 - KV versions, the Raft log, request results, and snapshot bytes live in the Raft-replicated store and its object-storage backups; only the shard directory, tenant quotas, group membership, and the snapshot catalog are relational.

```

CREATE TABLE tenant_quotas (
  tenant_id      text NOT NULL,
  max_keys       bigint NOT NULL,
  max_bytes      bigint NOT NULL,
  max_value_bytes integer NOT NULL DEFAULT 65536,    -- 64 KB request validation limit
  write_rate_limit integer NOT NULL,                -- commands per second admitted at the router
  watch_limit     integer NOT NULL,                  -- concurrent watch streams
  state          text NOT NULL DEFAULT 'ACTIVE',
  PRIMARY KEY (tenant_id),
  CHECK (state IN ('ACTIVE', 'SUSPENDED')),
  CHECK (max_keys > 0 AND max_bytes > 0 AND max_value_bytes > 0 AND write_rate_limit >= 0)
);

CREATE TABLE raft_groups (
  group_id       text NOT NULL,
  membership_epoch bigint NOT NULL DEFAULT 0,        -- bumps on every joint-consensus change
  voters         jsonb NOT NULL,                     -- node ids across 3 fault domains
  learners       jsonb NOT NULL DEFAULT '[]',
  leader_hint    text,                               -- routing hint only; authority is the read barrier
  state          text NOT NULL DEFAULT 'ACTIVE',
  PRIMARY KEY (group_id),
  CHECK (state IN ('BOOTSTRAPPING', 'ACTIVE', 'DRAINING', 'RETIRED')),
  CHECK (membership_epoch >= 0)
);

CREATE TABLE shard_directory (
  range_start    bytea NOT NULL,                     -- tenant-prefixed key bytes; inclusive
  ownership_epoch bigint NOT NULL,                   -- fences the old write owner on transfer
  range_end      bytea NOT NULL,                     -- exclusive
  group_id       text NOT NULL,
  state          text NOT NULL DEFAULT 'ACTIVE',
  target_group_id text,                              -- destination during TRANSFERRING
  updated_at     timestampz NOT NULL DEFAULT now(),
  PRIMARY KEY (range_start, ownership_epoch),
  FOREIGN KEY (group_id) REFERENCES raft_groups (group_id),
  FOREIGN KEY (target_group_id) REFERENCES raft_groups (group_id),
  CHECK (state IN ('ACTIVE', 'TRANSFERRING', 'RETIRED')),
  CHECK (range_start < range_end),
  CHECK (ownership_epoch >= 0)
);

CREATE UNIQUE INDEX shard_directory_current ON shard_directory (range_start)
  WHERE state <> 'RETIRED';                          -- one writable owner per range
CREATE INDEX shard_directory_by_group ON shard_directory (group_id) WHERE state <> 'RETIRED';

CREATE TABLE snapshot_catalog (
  group_id       text NOT NULL,
  last_included_index bigint NOT NULL,
  term           bigint NOT NULL,
  membership_epoch bigint NOT NULL,
  compacted_revision bigint NOT NULL,                -- cursors below this get the restart-required error
  snapshot_ref   text NOT NULL,                      -- object-storage key; state, dedup results,
  membership     text NOT NULL,
  checksum       text NOT NULL,                      -- verified before any log truncation
  created_at     timestampz NOT NULL DEFAULT now(),
  PRIMARY KEY (group_id, last_included_index, term),
  FOREIGN KEY (group_id) REFERENCES raft_groups (group_id),
  CHECK (last_included_index >= 0 AND term >= 0 AND compacted_revision >= 0)
);

CREATE INDEX snapshot_catalog_latest ON snapshot_catalog (group_id, last_included_index DESC); --
recovery scan

```

- Not captured by the DDL
 - KV version (tenant, key, revision), the Raft log (groupId, logIndex), and Request result (tenant, clientId, requestId) are state-machine rows inside each group; no SQL table can hold them without a second consistency authority.

- The current-key index, watch history, and TTL eligibility live in the replicated state machine and its compaction, not in these tables.
- The directory epoch is authoritative only when the router revalidates it against the group; a cached row is a hint.
- Deviation from the companion
 - The companion models a `Namespace catalog (tenant, namespace)`; this file scopes keys by tenant without a namespace level, so the quota row is keyed by `tenant_id` alone.
 - The companion pseudocode lists the MVCC record, replay, and watch rows; they are deliberately excluded here because they are not relational in this design.

Quorum commit and linearizable reads — IC3 baseline

- Mutation order
 - The leader appends a command to its durable log.
 - Followers persist entries before acknowledging replication according to the storage contract.
 - A majority replication count directly advances commit for entries from the leader's current term.
 - Committing a current-term entry also commits the preceding prefix.
 - Apply committed commands in order and return the resulting revision.
- Read authority
 - A node calling itself leader may have been partitioned while another leader was elected.
 - Establish current leadership through a quorum-confirmed read barrier such as `ReadIndex`.
 - Wait until the state machine has applied through that index before answering.
 - After election, establish a committed entry in the current term before relying on the optimized read path.
 - Lease-based reads need explicit bounded-clock assumptions and lease safety rules.
- CAS
 - Compare the expected revision inside the replicated state machine.
 - Do not read locally and later submit an unconditional write.
 - Two competing CAS commands are ordered, so the second observes the first's changed revision.
- Ambiguous timeout
 - The write may have committed even if the client saw no response.
 - Persist request identity and result with the command application.
 - Retry the same identity and reject a different command hash.
 - After deduplication retention expires, the API cannot promise replay of the original result.
- Why quorum arithmetic alone is insufficient
 - $R + W > N$ gives intersecting replica sets under its assumptions.
 - It does not by itself order concurrent writes or make read-modify-write atomic.
 - Timestamp last-write-wins can discard a valid concurrent update.

Snapshots, watches, and rebalancing — IC3 baseline

- MVCC reads
 - A list operation chooses one committed snapshot revision.
 - Bind pagination tokens to tenant, range, and revision.
 - Reject a cursor after its snapshot is compacted rather than mixing revisions.
- Watches
 - Deliver ordered committed events with revision cursors.
 - A disconnected watcher resumes from its last applied revision.
 - After compaction, obtain a snapshot at revision R and watch after R using the API's cursor convention.
 - A watch event does not prove that no newer write has completed.
- TTL
 - A leader proposes expiration as a replicated delete command.

- Replicas do not independently remove keys based on local wall clocks.
- TTL is an eligibility time, with a declared scheduling delay bound.
- Snapshot and compaction
 - Capture a consistent applied index with its term and membership configuration.
 - Persist and verify the snapshot before truncating required log history.
 - Retain deduplication results needed by the supported retry horizon.
 - Lagging replicas install a snapshot when their log gap has been compacted.
- Membership and shard movement
 - Add a learner and catch it up before changing voting membership.
 - Use the consensus protocol's safe configuration-change procedure.
 - A shard transfer changes the authoritative directory epoch and fences the old write owner.
 - Gossip can help discovery but cannot authorize two writable shard copies.
- Tombstone retention
 - Garbage-collect only after snapshot and replication watermarks make older values unreachable.
 - A forgotten replica must install a current snapshot before serving again.

Failure, operations, and security

Failure	Blast radius	Detection	Degraded behavior	Recovery
Leader crash after commit before reply	One shard client	Election and timeout	Retry same request identity	New leader replays committed result
One replica unavailable	One group	Replication lag	Quorum side continues	Catch up log or install snapshot
Quorum lost	One shard	Failed quorum confirmation	Reject authoritative reads and writes	Restore a safe majority; no forced stale promotion
Disk or snapshot corruption	One replica	Checksums and snapshot verification	Remove replica from serving path	Restore verified snapshot from healthy source
Watch cursor compacted	Affected watchers	Explicit compaction response	No silent event continuity claim	Snapshot and resume
Router holds a stale ownership epoch	Requests to one moved range	Group rejects the old epoch	Retry after directory refresh; no dual writers	Directory transfer fences the old group
Shard directory store unavailable	New routing lookups and shard moves	Directory health check	Routers serve from cached ownership; existing groups keep serving	Restore directory quorum; resume transfers from the fenced epoch
One tenant floods writes or watches	One shard and its co-located tenants	Per-tenant rate and watch-count metrics	Throttle that tenant at the router	Move the tenant's range to its own group
Fsync tail latency spike	One group's writes	Commit latency p99	Writes slow but stay durable; ReadIndex reads unaffected	Replace the slow disk or replica; transfer leadership

- Observe
 - Quorum commit latency and fsync tail latency.
 - Leader changes and apply lag.
 - MVCC size and compaction lag.
 - Per-tenant hot keys and watch fan-out.
- Security
 - Encrypt replica traffic and backups with separate recovery authorization.
 - Authorize each operation against the tenant's compartment policy before routing.
- Retry discipline
 - Exponential backoff with jitter on leader-unknown and quorum-timeout errors.
 - Retry an ambiguous write only with the same `requestId` and command hash.

- Bound attempts within the deduplication retention horizon.
- A 409 revision mismatch is a corrective action for the caller, not a retry.
- Tenant isolation
 - Enforce tenant prefixes from authenticated identity.
 - Limit transaction bytes and watch count.
 - Quota per-tenant write rate, key count, and watch fan-out so one tenant cannot starve a shared group.
 - A tenant's token receives 404 on another tenant's keys, not 403, to avoid existence leaks.
 - Move a tenant's range to its own group when its quotas keep firing.
- Rollout and rollback safety
 - Upgrade one replica at a time while preserving quorum.
 - Keep log-command and snapshot formats compatible across the rolling window.
 - Enable a new command format only after every replica runs the new version, so a rollback never meets an unreadable entry.
 - Canary one group before the fleet and keep leadership off canary replicas until commit latency matches.
 - Drill restore without reusing a lower external fencing epoch.

Alternatives and IC4 stretch

Option	What it is	Strength	Weakness	Choose when
Raft-backed shard	Replicated ordered state machine	Linearizable CAS and failover	Quorum latency; minority unavailable	Baseline strong metadata
Leaderless Dynamo-style store	Independent versions and repair	Partition write availability	Conflict resolution; no baseline atomic CAS	Application accepts eventual convergence
Managed relational database	Transactions and indexed records	Mature constraints and operations	May exceed needed data model or scale boundary	Small metadata service with no custom storage requirement

- Recommendation
 - Prefer a proven store such as etcd or a suitable managed database unless the prompt asks to build the internals.
- Stronger requirement
 - Cross-shard atomic transactions require an explicit coordinator and recovery protocol.
 - Cross-region linearizability places quorum traffic on the WAN.
- At 10× load
 - Add independent groups and distribute tenant ranges.
 - Keep hot-key limits visible instead of promising unlimited horizontal scale.
- Cost and blast radius
 - More small groups improve isolation but increase elections and background IO.
- Migration
 - Copy a snapshot, stream the tail, then switch ownership with a fenced epoch.
- Weakest assumption
 - Independent crash-fault replicas and storage that honors durable writes.
- Over-engineering warning
 - Do not invent a consensus algorithm or claim leaderless last-write-wins meets a CAS contract.

Level signal — IC3 baseline vs IC4 stretch

Phase	IC3 must land	IC4 adds	Separating signal
Scope	Define linearizability and transaction boundary	Negotiate WAN and cross-shard semantics	A precise consistency contract

Phase	IC3 must land	IC4 adds	Separating signal
Capacity	Size writes, history, and hot keys	Shard without hiding skew	No generic horizontal-scale claim
Correctness	Current-term commit and read authority	Safe membership and movement	Explains stale-leader failure
Recovery	Verified snapshots and cursor gaps	External epoch continuity on restore	No silent history rollback
Architecture	Router, Raft group, MVCC state machine, snapshots, one trace	Cost of each split and the fencing the directory epoch creates	Every component has a mechanism and a price
Wrap	Recap with quorum latency as the price	Names crash-only replicas and honest fsync as the weakest assumption	Identifies the flaw before the interviewer does

Interview pushback and validation

- Q1: Why not read any replica?
 - **IC3 baseline**
 - It may lag a completed write and violate linearizability.
 - **IC4 stretch**
 - Offer explicitly stale reads only for callers that accept the weaker contract.
- Q2: Does majority replication always let a leader commit an old-term entry directly?
 - **IC3 baseline**
 - No
 - Raft's direct majority rule requires a current-term entry.
 - **IC4 stretch**
 - Explain how a committed current-term entry establishes the preceding prefix.
- Q3: Two CAS calls compare the same revision?
 - **IC3 baseline**
 - The state machine orders them and at most one can replace that revision.
 - **IC4 stretch**
 - Cross-shard comparisons require a transaction protocol beyond one log.
- Q4: Can watches replace linearizable Get?
 - **IC3 baseline**
 - No
 - Delivery can lag newer committed writes.
 - **IC4 stretch**
 - Use revision barriers where clients need a current snapshot plus change stream.
- Q5 (premise reversal): Writes must succeed on both sides of a partition?
 - **IC3 baseline**
 - That conflicts with this single authoritative linearizable contract.
 - **IC4 stretch**
 - Choose explicit conflict-resolving eventual semantics or narrow ownership so operations do not conflict.
- Q6: Why not build a custom KV store for the CA metadata?
 - **IC3 baseline**
 - A relational database may already provide the required transactions.
 - **IC4 stretch**
 - Build custom internals only when measured scale or an interview requirement justifies the operational cost.
- Q7 (premise reversal): Drop Raft and use one primary database with synchronous replicas; is consensus overkill here?
 - **IC3 baseline**

- A primary with synchronous replicas still needs an agreed failover decision, otherwise a partitioned old primary and a promoted replica both accept writes.
- That decision is the consensus problem again, now hidden inside the failover tooling.
- **IC4 stretch**
 - Accept a managed database with fenced failover when the metadata fits one node and the team owns no storage internals.
 - Keep the linearizable CAS and ownership-epoch contract identical so callers do not change if the store is later sharded.
- Validation scenarios
 - Run concurrent CAS histories through a linearizability checker.
 - Partition the old leader and ensure it cannot serve authoritative stale reads.
 - Crash after durable commit but before reply and replay the request ID.
 - Compact history during pagination and require a clear restart error.
 - Add and remove replicas under load while preserving quorum rules.
 - Restore a snapshot and verify deduplication and external epoch continuity.

Recall card

- “Strong metadata is an ordering contract.”
- “A stale leader needs proof before answering a read.”
- “Quorum intersection is not a complete concurrency protocol.”
- “Sharding and replication solve different problems.”
- Wrap-up script
 - “Each shard is an ordered replicated state machine with linearizable reads and CAS.”
 - “Durable majority replication and read barriers define the safety boundary.”
 - “Snapshots and revision-aware clients make compaction and recovery explicit.”
 - “The price is quorum latency and refusal to serve the minority side; my weakest assumption is crash-only replicas on storage that honors fsync, so I would test power-loss durability first.”

References

- [Ranked bank](#): prompt and provenance.
- [Role framing](#): IC3 baseline, IC4 stretch, and round clock.
- [OCI role framing](#): role decode and scoring context.
- [API/schema companion](#): related contract drills.
- Reference check: 2026-09-10.
 - Official documentation supports the mechanisms cited next to each link.
 - Capacity values and architecture choices remain proposed interview assumptions.
- [Raft paper](#): replicated log safety, commitment, and membership changes.
- [etcd API guarantees](#): linearizable KV operations and watch guarantees.
- [etcd Raft implementation](#): read barriers and storage responsibilities.